Networking and Ingress

Expose the application via Ingress.

Week 5 - Tasks

- Install Ingress Controller.
- Apply the `k8s/` manifest directory for this week to create new resources, including Ingress.
- Check if the Ingress Controller is running.
- Find out the NodePort and the Node IP on which the pod is running, then curl the application.
- Check if the frontend can access the backend and retrieve the API notes. (See the **Verify** section below).

Extra Challenges

1. **Gateway API is replacing Ingress API:**
    - Read up on why the Gateway API is considered a better alternative.
    - Can you implement the Gateway API on this project?
2. **Let's Encrypt Integration:**
    - Let's Encrypt needs to validate ownership of a domain if you have one. Can you implement TLS with Let's Encrypt on your ingress?
3. **HERCULEAN TASK:**
    - Expose the Kind ingress to your host/browser so that you can access the API at `notes.local/api` by adding `notes.local` to `/etc/hosts`.
    - *Note: You may have to make changes to the cluster. Best wishes!*

To Think About

- Why is the LoadBalancer in a `pending` state?

Solution Guide

- **Install Nginx Ingress Controller:**

  Bash

  ```
  kubectl apply -f https://raw.githubusercontent.com/kubernetes/ingress-nginx/controller-v1.8.1/deploy/static/provider/cloud/deploy.yaml
  ```

- **Apply Ingress Resource:**

  Bash

  ```
  kubectl apply -f k8s/
  ```

- **Check Ingress Routing is Ready and Working:**

  Bash

  ```
  kubectl wait --namespace ingress-nginx --for=condition=ready pod --
  selector=app.kubernetes.io/component=controller --timeout=120s
  ```

  - *Expected Result:* `pod/ingress-nginx-controller-some-random-alphanum condition met`
- **Find NodePorts:**

  Bash

  ```
  kubectl get svc -n ingress-nginx
  ```

- **Find Worker IP:**

  Bash

  ```
  kubectl get nodes -o wide
  ```

Verify

- **Curl the App:**

  Bash

  ```
  curl -v -H "Host: notes.local" http://<WORKER-NODE-IP>:<NODEPORT>/api
  ```

  You may have to try different combinations of Worker Node IP with the NodePort serving HTTP port 80.

  - *Expected Result:*

    HTTP

    ```
    < HTTP/1.1 200 OK
    < Date:
    < Content-Type: application/json
    < Content-Length: 87
    < Connection: keep-alive
    <
    * Connection #0 to host <WORKER-NODE-IP> left intact
    {"message":"Cloud Native
    Notes","endpoints":["/health","/api/notes","/api/notes/{id}"]}
    ```

    Congratulations if you get an HTTP `200 OK` response and the API endpoints in JSON format! This shows your ingress is routing traffic to the cluster. Notice we no longer do port forwarding to access the backend.

- **Check Frontend Communication with Backend:**

Bash

```
kubectl exec -n cloud-native-notes <frontend-pod-xxxxxxxx> -- curl -i
http://backend:3000/api/notes
```

- **View the Frontend in Action:**

  Bash

  ```
  curl -v -H "Host: notes.local" <WORKER-NODE-IP>:<NODEPORT>
  ```